

TECHEVENT

A Luma-inspired Events Platform for Tech & Hackathons

Complete Project Documentation & Team Work Division

React + Node.js • Razorpay • WhatsApp & Email • QR Entry

3 Members

React+Node

Razorpay

QR Entry

Version 1.0 • May 2026

Table of Contents

1. Project Overview & Vision	3
2. Core Features	3
3. System Architecture	4
4. Tech Stack	4
5. Database Schema	5
6. API Endpoints	6
7. Razorpay Payment Integration	7
8. Email & WhatsApp Follow-ups	8
9. QR Code Entry System	9
10. Frontend Pages & UI	10
11. Team Work Division	11
12. Sprint Plan (4 Weeks)	14
13. Deployment Guide	15
14. Environment Variables	15

1. Project Overview & Vision

TechEvent is a full-stack web platform inspired by Luma, built specifically for the **college tech ecosystem** — hackathons, startup demo days, tech talks, product launches, and developer meetups. It lets organizers create beautiful event pages, accept paid/free registrations via Razorpay, and manage attendees with automated WhatsApp + email follow-ups and QR-based check-in at the door.

Target Audience	College students, hackathon organizers, startup founders, tech clubs
Primary Use-case	Create & manage tech events / hackathons with smooth attendee experience
Key Differentiator	India-first payments (Razorpay INR), WhatsApp automation, QR entry
Platform	Web (mobile-responsive React SPA + Node.js REST API)

2. Core Features

Event Mgmt

Event Creation & Management

Rich event pages with cover image, agenda, speaker cards, location (offline/online), capacity limits, custom registration fields, public/private toggle.

Payments

Razorpay Integration

INR ticket sales with multiple tiers (free / paid / donation). Razorpay Checkout embedded in registration flow. Auto-refunds on cancellation. Payment dashboard for organisers.

QR Entry

QR Code Check-in

Unique QR per attendee generated on registration. Organiser scans via mobile camera on entry. Real-time attendee count updates. Prevents duplicate entries.

Email

Automated Email Flows

Registration confirmation with QR attachment. Event reminder 24 h before. Post-event thank-you + feedback form. Waitlist notification. Built with Nodemailer + SendGrid.

WhatsApp

WhatsApp Follow-ups

Registration confirmation message with QR image. Day-before reminder. Day-of entry reminder. Built via Twilio WhatsApp API / WATI.

Discovery**Event Discovery**

Public homepage with upcoming tech events. Filter by city, date, tag (Hackathon / Talk / Workshop). Calendar view. Search.

Dashboard**Organiser Dashboard**

Attendee list with export (CSV/Excel). Real-time check-in stats. Revenue summary. Refund management. Edit event details.

Auth**Auth & Profiles**

Google OAuth + email/password. Attendee profile with past events. Organiser profile with created events. JWT sessions.

3. System Architecture

The system follows a classic **client-server architecture** with separation of concerns across frontend, backend, and third-party services.

Layer	Technology	Responsibility
Frontend	React 18 + Vite + TailwindCSS	SPA, routing, UI components, QR scanner
Backend API	Node.js + Express.js	REST API, business logic, auth middleware
Database	PostgreSQL + Prisma ORM	Persistent storage for events, users, tickets
Cache / Queue	Redis	Session cache, job queue for emails/WhatsApp
File Storage	Cloudinary	Event cover images, QR code image storage
Payments	Razorpay	Payment orders, webhook verification, refunds
Email	Nodemailer + SendGrid SMTP	Transactional email delivery
WhatsApp	Twilio WhatsApp API	WhatsApp message delivery
QR Engine	qrcode (npm)	Server-side QR generation as PNG/Base64
Auth	Passport.js + JWT	Google OAuth 2.0 + local strategy
Deployment	Render / Railway (API) + Vercel (FE)	CI/CD, environment management

4. Tech Stack

FRONTEND

- React 18 with Vite (build tool)
- React Router v6 (client routing)
- TailwindCSS + shadcn/ui (styling)
- Zustand (state management)
- React Query (server state / caching)
- html5-qrcode (QR scanner in browser)
- Axios (HTTP client)
- Framer Motion (animations)
- react-qr-code (display QR on ticket page)

BACKEND

- Node.js 20 LTS
- Express.js (REST framework)
- Prisma ORM (PostgreSQL)
- Passport.js + JWT (auth)
- Multer + Cloudinary (uploads)
- qrcode npm (generate QR PNGs)
- nodemailer + @sendgrid/mail
- twilio (WhatsApp messages)
- bull (Redis job queues)
- razorpay npm SDK
- zod (request validation)

DEVOPS/INFRA

- PostgreSQL 15 (primary DB)
- Redis 7 (cache + queues)
- Cloudinary (media CDN)
- Vercel (React deployment)
- Railway / Render (API + DB)
- GitHub Actions (CI/CD)
- dotenv (env management)

5. Database Schema (Prisma)

Key models and their fields. Foreign keys and indexes are managed by Prisma migrations.

model User {

```
id String @id @default(cuid())
email String @unique
name String
avatar String?
role Role @default(ATTENDEE) // ATTENDEE | ORGANIZER | ADMIN
googleId String? @unique
createdAt DateTime @default(now())
events Event[] // events created (if ORGANIZER)
tickets Ticket[]
}
```

model Event {

```
id String @id @default(cuid())
title String
description String
coverImage String?
startAt DateTime
endAt DateTime
location String? // address or 'Online'
meetLink String?
capacity Int?
isPublic Boolean @default(true)
tags String[] // ['Hackathon', 'AI', 'Startup']
organizerId String
organizer User @relation(fields:[organizerId], references:[id])
ticketTiers TicketTier[]
tickets Ticket[]
createdAt DateTime @default(now())
}
```

model TicketTier {

```
id String @id @default(cuid())
eventId String
name String // 'General' | 'Student' | 'VIP'
price Int // in paise (0 = free)
quantity Int
sold Int @default(0)
event Event @relation(...)
tickets Ticket[]
}
```

model Ticket {

```
id String @id @default(cuid())
userId String
eventId String
tierId String
qrCode String // unique token for QR
qrImageUrl String? // Cloudinary URL of QR image
status TicketStatus // CONFIRMED|CANCELLED|CHECKED_IN
paymentId String? // Razorpay payment_id
orderId String? // Razorpay order_id
createdAt DateTime @default(now())
checkedInAt DateTime?
user User @relation(...)
event Event @relation(...)
}
```


6. API Endpoints

Auth

Method	Endpoint	Description
POST	/api/auth/register	Register with email + password
POST	/api/auth/login	Login, returns JWT
GET	/api/auth/google	Initiate Google OAuth
GET	/api/auth/google/callback	Google OAuth callback
POST	/api/auth/logout	Invalidate token

Events

Method	Endpoint	Description
GET	/api/events	List public events (filters: tag, city, date)
POST	/api/events	Create event (ORGANIZER only)
GET	/api/events/:id	Get event details
PUT	/api/events/:id	Update event (owner only)
DELETE	/api/events/:id	Delete event (owner only)
GET	/api/events/:id/attendees	Attendee list (owner only)

Tickets & Payments

Method	Endpoint	Description
POST	/api/tickets/order	Create Razorpay order, returns order_id
POST	/api/tickets/verify	Verify payment signature, issue ticket
GET	/api/tickets/mine	All tickets of logged-in user
POST	/api/tickets/:id/cancel	Cancel ticket & initiate refund
POST	/api/webhooks/razorpay	Razorpay webhook handler (HMAC verified)

QR Check-in

Method	Endpoint	Description
--------	----------	-------------

GET	/api/checkin/:qrCode	Validate QR, mark CHECKED_IN
GET	/api/events/:id/checkin-stats	Live check-in count for organiser

Notifications

Method	Endpoint	Description
POST	/api/notify/test-email	Send test email (dev only)
POST	/api/notify/test-whatsapp	Send test WhatsApp (dev only)

7. Razorpay Payment Integration

TechEvent uses Razorpay's **Order API** for a two-step, server-verified payment flow. This prevents client-side tampering.

Step 1 – Create Order (Backend)

```
POST /api/tickets/order → validate ticket availability
Call Razorpay Orders API: razorpay.orders.create({ amount, currency:'INR', receipt })
Save order_id + amount in DB with status PENDING
Return { order_id, amount, key_id } to client
```

Step 2 – Open Checkout (Frontend)

```
Load Razorpay checkout.js script dynamically
Instantiate Razorpay({ key, amount, order_id, ... })
On payment success handler, get { razorpay_payment_id, razorpay_order_id,
razorpay_signature }
POST to /api/tickets/verify with these three values
```

Step 3 – Verify Signature (Backend)

```
const hmac = crypto.createHmac('sha256', RAZORPAY_SECRET)
hmac.update(order_id + '|' + payment_id)
If digest === razorpay_signature → payment valid
Generate unique qrCode token (uuid), create Ticket record
Trigger QR generation job, email job, WhatsApp job via Bull queue
```

Step 4 – Webhook (Backup verification)

```
Razorpay sends payment.captured event to /api/webhooks/razorpay
Verify X-Razorpay-Signature header with webhook secret
Idempotently confirm ticket if not already confirmed
```

Refunds

```
POST /api/tickets/:id/cancel → organiser or user triggers cancel
Call razorpay.payments.refund(payment_id, { amount })
Update ticket status to CANCELLED, notify user via email + WhatsApp
```

Key Razorpay npm usage:

```
const Razorpay = require('razorpay');
const rzp = new Razorpay({ key_id: process.env.RAZORPAY_KEY_ID,
key_secret: process.env.RAZORPAY_SECRET });
const order = await rzp.orders.create({ amount: price*100, currency:'INR',
receipt: `rcpt_${ticketId}` });
```

8. Email & WhatsApp Follow-ups

8.1 Email — Nodemailer + SendGrid

All emails are sent via SendGrid SMTP through Nodemailer. A Bull queue handles retries and rate limiting.

Email Flow	Trigger	Content
Registration Confirmation	Immediately after payment verify	Ticket attached as QR image PNG, event details, calendar .ics file
24-hour Reminder	Cron job 24 h before event startAt	Event location, agenda, what to bring, entry QR reminder
Day-of Entry Reminder	2 h before event startAt	QR image, venue map link, check-in opens message
Post-event Thank You	1 h after event endAt	Feedback form link, photos link, next events suggestion
Waitlist Notification	When spot opens up	Invite to complete registration with deadline
Cancellation Confirmation	On ticket cancel	Refund timeline, amount refunded confirmation

8.2 WhatsApp — Twilio WhatsApp API

Twilio's WhatsApp Business API is used for conversational follow-ups. Messages use pre-approved WhatsApp templates.

WA Flow	Trigger	Template Preview
Registration Confirm	On ticket issue	Hi {name}! Your ticket for {event} is confirmed. QR Code: [image]
24-h Reminder	Cron 24 h before	Reminder: {event} is tomorrow at {time}. Venue: {location}
Day-of Check-in Alert	2 h before	Today's the day! Show this QR at entry: [image]
Cancellation	On cancel	Your ticket for {event} has been cancelled. Refund in 5-7 days.

Key implementation snippet (Twilio):

```
const twilio = require('twilio');
const client = twilio(process.env.TWILIO_SID, process.env.TWILIO_TOKEN);
await client.messages.create({
  from: 'whatsapp:+14155238886', // Twilio sandbox or approved number
  to: `whatsapp:${userPhone}`,
  body: `Hi ${name}! Your ticket for ${eventTitle} is confirmed.`,
  mediaUrl: [qrImageCloudinaryUrl], // sends QR image inline
});
```

9. QR Code Entry System

Each ticket gets a cryptographically unique QR token. The organiser uses any modern mobile browser to scan and validate entry — no dedicated app needed.

QR Generation (Backend)

```
import QRCode from 'qrcode';
const token = `TKT-${ticket.id}-${crypto.randomUUID()}`;
// Encode token as QR PNG buffer
const qrBuffer = await QRCode.toBuffer(token, { errorCorrectionLevel:'H', width:400 });
// Upload to Cloudinary
const result = await cloudinary.uploader.upload_stream({ folder:'qrCodes' }, ...);
// Save qrCode token + qrImageUrl to Ticket in DB
```

QR on Ticket Page (Frontend)

```
// Attendee opens /my-tickets/:ticketId
// Backend returns ticket.qrImageUrl OR ticket.qrCode token
// Frontend renders: <img src={ticket.qrImageUrl} /> for download
// Alt: render locally with: <QRCodeSVG value={ticket.qrCode} size={256} />
```

Scanner Page — Organiser Mobile (Frontend)

```
// Route: /events/:id/checkin (protected, organiser only)
// Uses html5-qrcode library to access camera
import { Html5QrcodeScanner } from 'html5-qrcode';
// On scan success: POST /api/checkin/:decodedQrToken
// Show green tick (valid) or red X (already scanned / invalid)
```

Check-in Validation (Backend)

```
// GET /api/checkin/:qrCode
const ticket = await prisma.ticket.findUnique({ where:{ qrCode } });
if (!ticket) return res.status(404).json({ valid:false, msg:'Invalid QR' });
if (ticket.status==='CHECKED_IN') return res.json({ valid:false, msg:'Already scanned' });
await prisma.ticket.update({ where:{qrCode}, data:{ status:'CHECKED_IN', checkedInAt:new Date() }});
return res.json({ valid:true, attendeeName: ticket.user.name });
```

10. Frontend Pages & UI

Route	Page Name	Content / Purpose
/	Home / Discover	Hero + trending events grid, filter bar (tag / city / date), search
/create	Create Event	Multi-step form: basic info → ticket tiers → cover image → publish
/events/:id	Event Detail	Cover, title, date, location, agenda, speakers, ticket CTA, map
/events/:id/register	Registration	Tier selection, attendee details form, Razorpay checkout
/my-tickets	My Tickets	List of user's upcoming & past tickets
/my-tickets/:id	Ticket Detail	QR code display, event info, download QR, cancel option
/dashboard/:eventId	Organiser Dashboard	Attendee table, check-in stats, revenue, export CSV
/events/:id/checkin	QR Scanner	Camera feed with html5-qrcode, live attendee counter
/profile	User Profile	Edit name, phone (for WhatsApp), past events
/auth/login	Login	Email+password form + Google OAuth button
/auth/register	Register	Sign-up form with phone number (for WhatsApp)

UI Design Notes (Luma-inspired):

- Dark gradient hero (#0F0F1A to #1A1A2E) with purple accent (#6C47FF)
- Event cards: glassmorphism style, hover elevation, tag pills
- Mobile-first; the QR scanner page is designed for portrait mobile
- Smooth page transitions using Framer Motion
- Cover image: 16:9 aspect ratio, Cloudinary auto-optimized

11. Team Work Division

The project is split into three focused roles. Each member owns a full vertical slice so pull request conflicts are minimal.

Member 1

Frontend Engineer — UI & Attendee Experience

Core Ownership: All React pages visible to attendees. The visual polish, responsiveness, and attendee-facing UX is entirely this member's responsibility.

Project Setup & Routing

- Scaffold Vite + React project, configure TailwindCSS + shadcn/ui
- Set up React Router v6 with protected routes (auth guard HOC)
- Create global Zustand store: auth slice, event slice
- Set up Axios instance with JWT interceptor (auto-attach Bearer token)
- Configure React Query provider for server state

Home & Discovery Page (/)

- Hero section with animated background gradient
- EventCard component (cover image, title, date, location, tag pill, price badge)
- FilterBar component: tag buttons, city dropdown, date picker
- Responsive event grid (3 col desktop → 2 tablet → 1 mobile)
- Infinite scroll / pagination using React Query
- Search input with 300 ms debounce

Event Detail Page (/events/:id)

- Cover image hero with overlay
- Sticky right sidebar: ticket CTA, countdown timer
- Agenda accordion, Speaker cards (name, photo, bio)
- Location section with embedded Google Maps iframe
- Related events row at bottom

Registration Page (/events/:id/register)

- Ticket tier selection cards (free vs paid, quantity picker)
- Attendee details form (name, email, phone, college for hackathons)
- Razorpay checkout.js integration: call backend /api/tickets/order,
- open Razorpay modal, on success call /api/tickets/verify
- Loading/success/failure states with toast notifications

My Tickets & Ticket Detail

- List view of all user tickets grouped by upcoming / past
- Ticket detail: QR image display using react-qr-code, event mini-card
- Download QR as PNG button (canvas toBlob)
- Cancel ticket flow with confirmation dialog

Auth Pages — Login & Register

- Login form with email/password + 'Sign in with Google' button
- Register form including phone number field for WhatsApp
- Form validation with react-hook-form + zod schemas
- JWT stored in memory (Zustand), refresh token in httpOnly cookie

Components & Design System

- Button, Input, Modal, Toast, Badge, Spinner, Avatar, Dropdown
- Framer Motion page transition wrapper
- Dark/light mode toggle (TailwindCSS class strategy)
- Responsive navbar: desktop link row + mobile hamburger sheet

Member 2

Backend Engineer — API, Auth & Database

Core Ownership: The entire Node.js/Express API, database schema, authentication, and organiser dashboard backend. Member 2 is also responsible for QR generation logic.

Project Setup & Infrastructure

- Scaffold Express app with folder structure: routes/ controllers/ services/ middleware/ prisma/
- Set up Prisma with PostgreSQL: schema.prisma, migrations
- Configure helmet, cors, rate-limiter-flexible, morgan logger
- Connect Redis with ioredis for caching and Bull queue
- Set up Cloudinary SDK for media uploads

Authentication System

- Implement Passport.js local strategy (bcrypt password hashing)
- Implement Google OAuth 2.0 strategy with Passport
- JWT signing & verification middleware (access token 15 min, refresh 7 days)
- POST /auth/register, POST /auth/login, GET /auth/google, GET /auth/google/callback
- Auth guard middleware: requireAuth(roles) for route protection

Event CRUD API

- Full CRUD for /api/events with organiser ownership checks

- File upload endpoint: multer → Cloudinary for cover image
- Tag & city filtering with Prisma where clauses
- Capacity management: prevent registration beyond capacity
- GET /api/events/:id/attendees — paginated, search by name

QR Code Generation

- npm install qrcode; generate QR as PNG buffer on ticket confirm
- Upload QR PNG to Cloudinary folder 'qrcodes'
- Store { qrCode: token, qrImageUrl } on Ticket record
- GET /api/checkin/:qrCode — atomic check-in with Prisma transaction
- GET /api/events/:id/checkin-stats — real-time count for organiser

Organiser Dashboard API

- GET /api/events/:id/attendees (paginated, filterable, with CHECKED_IN flag)
- GET /api/events/:id/revenue (sum of confirmed ticket payments)
- GET /api/events/:id/export (returns CSV stream of attendee list)
- PUT /api/events/:id (update event — only by owner)
- Real-time check-in count cached in Redis (invalidate on each scan)

Database & Migrations

- Write all Prisma schema models (User, Event, TicketTier, Ticket, Waitlist)
- Seed script: create sample events, users, ticket tiers for dev
- Add DB indexes: events.startAt, tickets.qrCode (unique), users.email
- Prisma migration management: npx prisma migrate dev

Testing & Validation

- Zod schemas for all request bodies (validated in middleware)
- Unit tests for auth middleware, QR validation logic (Jest)
- Integration tests for event CRUD and check-in endpoint
- Postman collection with all endpoints + environment variables

Core Ownership: Razorpay payment flow (both ends), all email & WhatsApp notification services, the QR scanner UI, and the organiser dashboard frontend. This member bridges backend services with the organiser-facing experience.

Razorpay Payment Integration (Backend side)

- npm install razorpay; configure Razorpay SDK with key_id + secret
- POST /api/tickets/order: validate tier, create Razorpay order, return order_id
- POST /api/tickets/verify: HMAC signature verification, issue ticket on success
- POST /api/webhooks/razorpay: verify X-Razorpay-Signature, idempotent confirmation
- POST /api/tickets/:id/cancel: call razorpay.payments.refund(), update DB

Bull Job Queues (Notifications Pipeline)

- Set up Bull queues: emailQueue, whatsappQueue
- Queue a job immediately after ticket confirm: { type, ticketId, userId }
- Queue cron reminders: use Bull's repeatable jobs or node-cron for 24h / 2h jobs
- Implement job processors: emailQueue.process(async job => { ... })
- Retry config: 3 attempts, exponential backoff on failure

Email Service (Nodemailer + SendGrid)

- npm install nodemailer @sendgrid/mail; configure SMTP transport
- Create HTML email templates (inline CSS) for each flow
- Registration confirm: embed QR image as CID attachment
- Attach calendar .ics file to registration email
- 24h reminder, day-of reminder, post-event, cancellation emails
- Handle email bounce/failure logging

WhatsApp Service (Twilio)

- npm install twilio; configure with TWILIO_SID + TWILIO_TOKEN
- Register WhatsApp message templates with Twilio (pre-approval required)
- Registration confirm: send message + media (qrImageUrl from Cloudinary)
- 24h reminder, day-of, cancellation messages
- Store whatsapp_message_sid in DB for tracking delivery status
- Webhook: receive Twilio delivery receipts, log to DB

QR Scanner Page (Frontend — /events/:id/checkin)

- Build route protected to ORGANIZER role

- Integrate html5-qrcode: camera permission request, scanner UI
- On successful decode: POST to /api/checkin/:token
- Show result: green card (name, time) for valid, red card for invalid/duplicate
- Live counter: poll GET /api/events/:id/checkin-stats every 5 s
- Offline resilience: queue scans in localStorage if network drops, flush on reconnect

Create Event Form (Frontend — /create)

- Multi-step form wizard (step 1: info, step 2: tiers, step 3: image, step 4: review)
- Cover image upload with drag-and-drop (react-dropzone) + crop (react-image-crop)
- Ticket tier builder: add/remove tier rows, set price (0 for free), quantity
- Rich text editor for description (using TipTap or Quill)
- Publish / Save as Draft toggle
- POST /api/events with FormData (image + JSON fields)

Organiser Dashboard (Frontend — /dashboard/:eventId)

- Attendee data table with columns: name, email, tier, payment, check-in status
- Search + filter (checked-in vs not) within the table
- Export CSV button: triggers GET /api/events/:id/export download
- Revenue card: total collected INR, breakdown by tier
- Real-time check-in progress bar: checked-in / total
- Quick links: Edit Event, View Public Page, Open Scanner

12. Sprint Plan — 4 Weeks

Week 1 — Foundation

M1

Project scaffold, routing, TailwindCSS, auth pages (login/register), Navbar, global layout

M2

Express scaffold, Prisma schema + migrations, auth API (local + Google OAuth), JWT middleware

M3

Razorpay account setup, Twilio sandbox setup, SendGrid API key, environment config, Bull queue skeleton

Week 2 — Core Event & Payment

M1

Home/Discovery page, EventCard component, FilterBar, Event Detail page

M2

Events CRUD API, Cloudinary cover image upload, attendee capacity logic, QR generation on ticket

M3

Razorpay order + verify endpoints, webhook handler, Registration page frontend with Razorpay checkout

Week 3 — Notifications, QR Entry, Dashboard

M1

My Tickets page, Ticket Detail with QR display, Cancel ticket flow

M2

Check-in API (/api/checkin/:qrCode), stats endpoint, CSV export, organiser attendee list API

M3

Email flows (confirm, reminder, cancel), WhatsApp flows, QR Scanner page, Create Event form + Organiser Dashboard

Week 4 — Polish, Testing & Deploy

M1

Animations (Framer Motion), mobile responsiveness audit, empty states, error boundaries, loading skeletons

M2

Unit + integration tests, Postman collection, DB indexes + performance, seed data

M3

End-to-end payment flow testing, notification delivery testing, deploy frontend (Vercel) + backend (Railway)

13. Deployment Guide

Service	Platform	Config Notes
React Frontend	Vercel	Connect GitHub repo, set VITE_API_URL env var, auto-deploy on push to main
Node.js API	Railway / Render	Dockerfile or nixpacks, set all env vars in dashboard, expose port 8080
PostgreSQL	Railway (Postgres plugin)	Copy DATABASE_URL from Railway dashboard into API env vars
Redis	Railway (Redis plugin)	Copy REDIS_URL from Railway dashboard into API env vars
Cloudinary	cloudinary.com	Free tier sufficient. Get CLOUD_NAME, API_KEY, API_SECRET
Media CDN	Cloudinary CDN	Auto-served via res.cloudinary.com — no extra config needed

14. Environment Variables

All secrets are managed via .env files. Never commit these to Git — add .env to .gitignore.

Database & Cache

```
DATABASE_URL=postgresql://user:pass@host:5432/techemvent
REDIS_URL=redis://localhost:6379
```

Auth

```
JWT_SECRET=your-super-secret-32-char-string
JWT_REFRESH_SECRET=another-secret-for-refresh
GOOGLE_CLIENT_ID=from Google Cloud Console OAuth 2.0
GOOGLE_CLIENT_SECRET=from Google Cloud Console OAuth 2.0
GOOGLE_CALLBACK_URL=https://yourapi.com/api/auth/google/callback
```

Razorpay

```
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxxxx
RAZORPAY_SECRET=your-razorpay-secret
RAZORPAY_WEBHOOK_SECRET=from Razorpay Dashboard > Webhooks
```

Email (SendGrid)

```
SENDGRID_API_KEY=SG. xxxxxxxxxxxxxx
EMAIL_FROM=noreply@yourdomain.com
```

WhatsApp (Twilio)

```
TWILIO_SID=ACxxxxxxxxxxxxx
TWILIO_TOKEN=your-auth-token
TWILIO_WHATSAPP_NUMBER=+14155238886
```

Cloudinary

```
CLOUDINARY_CLOUD_NAME=your-cloud-name
```

```
CLOUDINARY_API_KEY=your-api-key
```

```
CLOUDINARY_API_SECRET=your-api-secret
```

Frontend (.env in Vite project)

```
VITE_API_URL=https://yourapi.railway.app
```

```
VITE_RAZORPAY_KEY_ID=rzp_test_XXXXX (public key only)
```

```
VITE_GOOGLE_CLIENT_ID=your-google-client-id
```